

Platform iOS | Android | Web

Backend NestJS 10

Mobile Expo SDK 54

Database PostgreSQL 16

License Private



IDGate — Identification Gate

A unified digital identity platform that connects Natural Persons, Legal Entities, and Virtual Characters (Positions) through a secure, privacy-first identification system.



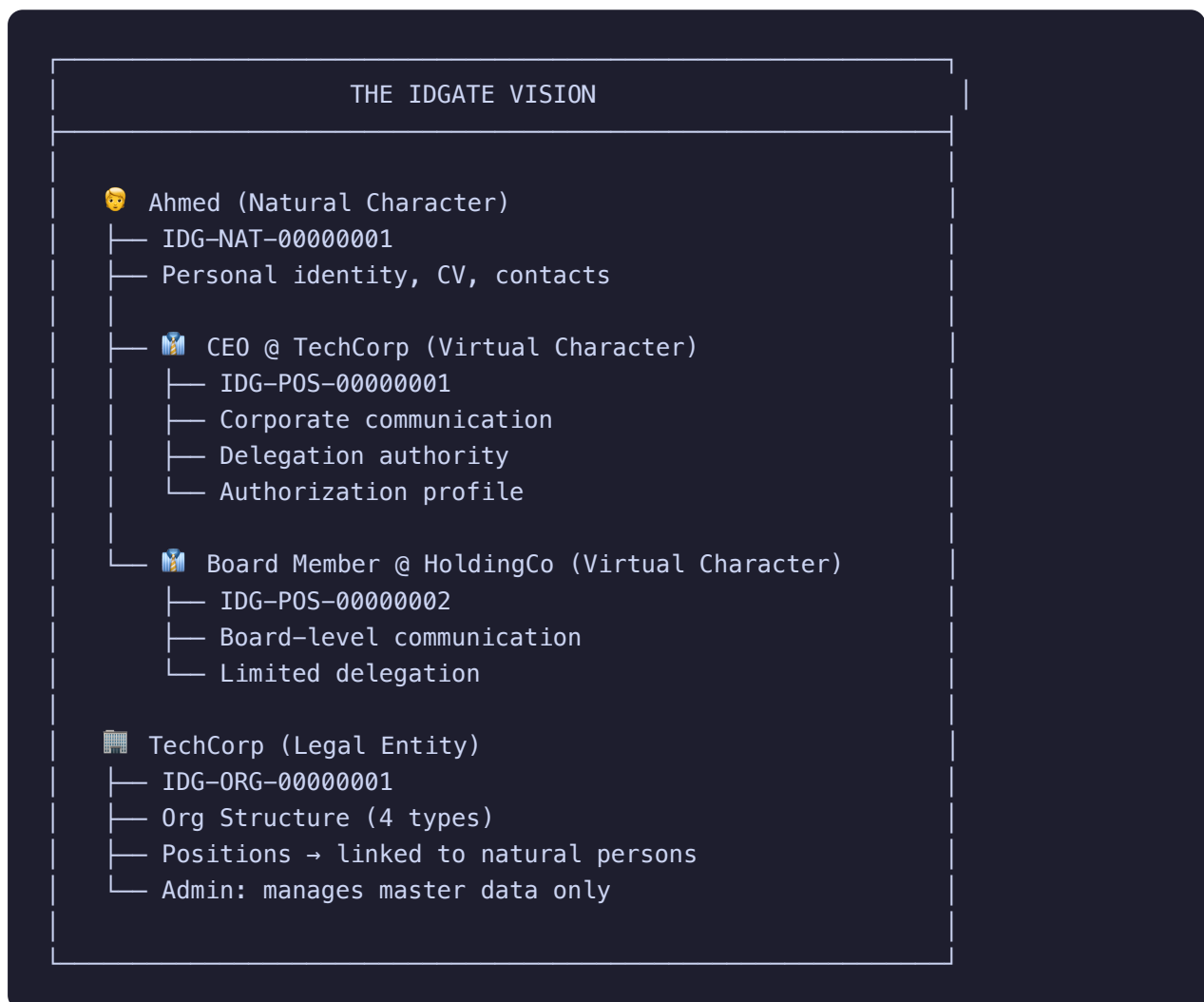
Table of Contents

- The Idea
 - Core Concepts
 - System Architecture
 - Three Character Types
 - Organization Structure System
 - Position Lifecycle
 - Communication Model
 - Privacy & Authorization
 - API Reference
 - Database Schema
 - Project Structure
 - Tech Stack
 - Getting Started
 - Deployment
 - Environment Variables
-

The Idea

IDGate (Identification Gate) revolutionizes how individuals and organizations establish, verify, and manage their digital identities. Unlike traditional systems where a person has one account, IDGate recognizes that people operate in **multiple capacities**:

- As **themselves** (Natural Character) — personal identity
- As an **employee/officer** (Virtual Character) — position-bound identity
- As an **organization admin** (Legal Entity) — corporate identity management



Key Principles

1. **Separation of Identities** — Personal communication is separate from professional
2. **Position-Based Access** — Corporate rights attach to positions, not people
3. **Privacy by Design** — Granular privacy controls per data category

4. **Delegation Chain** — Authority flows from CEO through positions
5. **Universal IDCode** — Every entity gets a unique IDG-XXX-XXXXXXXX code

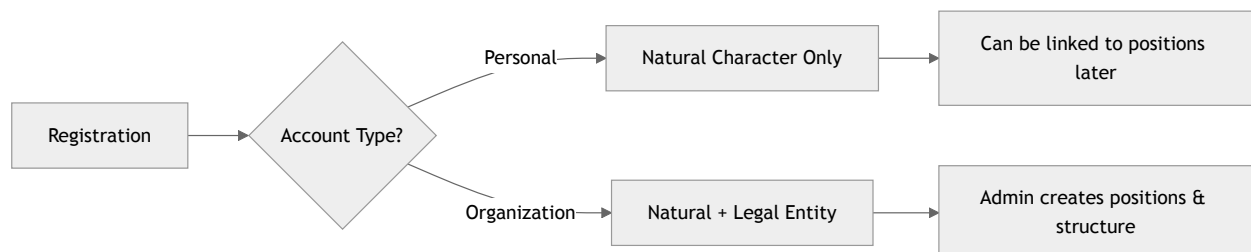
Core Concepts

IDCode System

Every entity in the system receives a universally unique identification code:

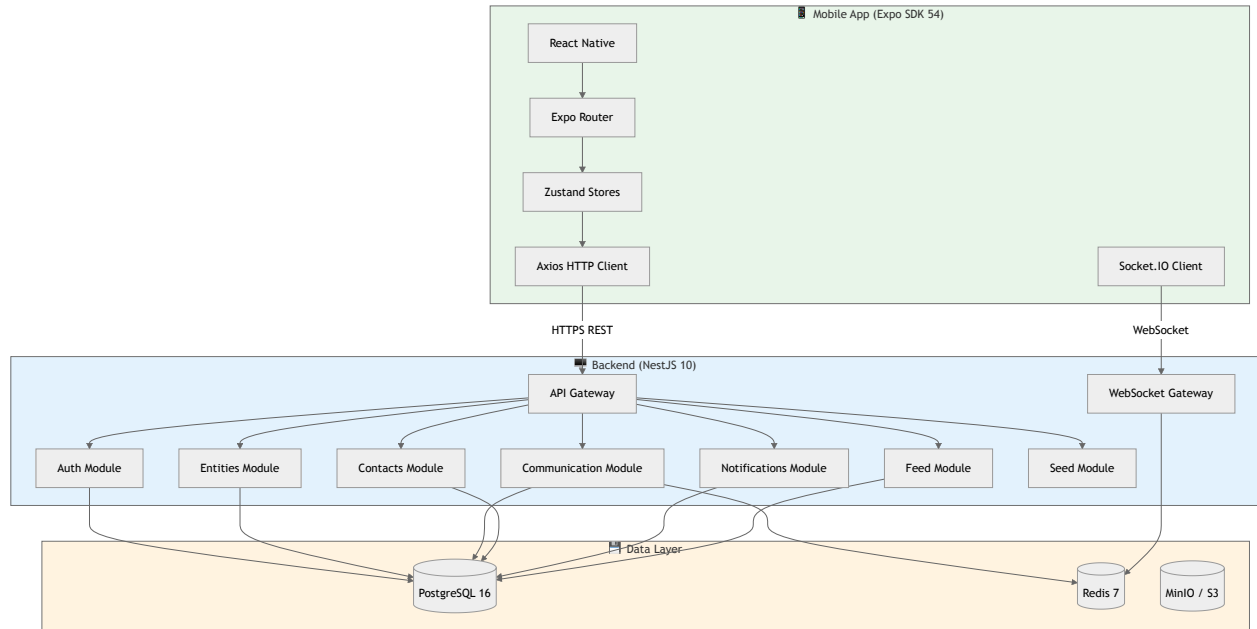
Entity Type	Format	Example
Natural Character (Person)	IDG-NAT-XXXXXXXX	IDG-NAT-00000001
Legal Entity (Organization)	IDG-ORG-XXXXXXXX	IDG-ORG-00000001
Virtual Character (Position)	IDG-POS-XXXXXXXX	IDG-POS-00000001

Account Types

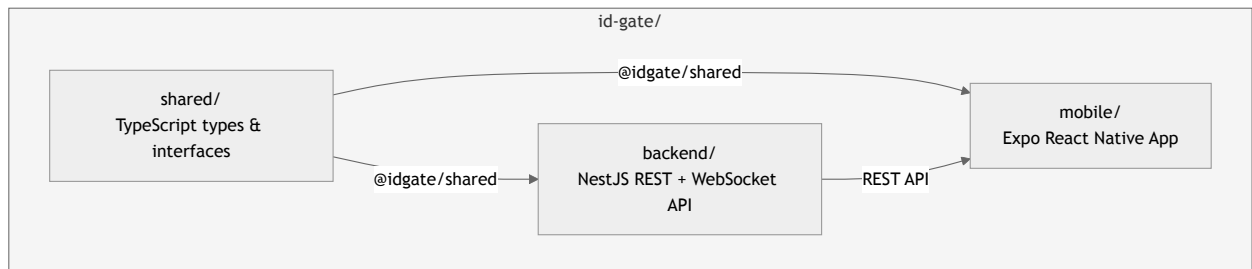


System Architecture

High-Level Architecture



Monorepo Structure



Three Character Types

1. Natural Character (الشخصية الطبيعية)

The **real person** — their true identity with personal data.



Natural Character

Personal Info

Name, Gender, DOB,
Nationality 1-4

Documents

National ID, Passports 1-4,
Driving License

Languages

Mother Tongue + 4
Languages w/ Proficiency

Contacts

Phone, Email, LinkedIn,
WhatsApp

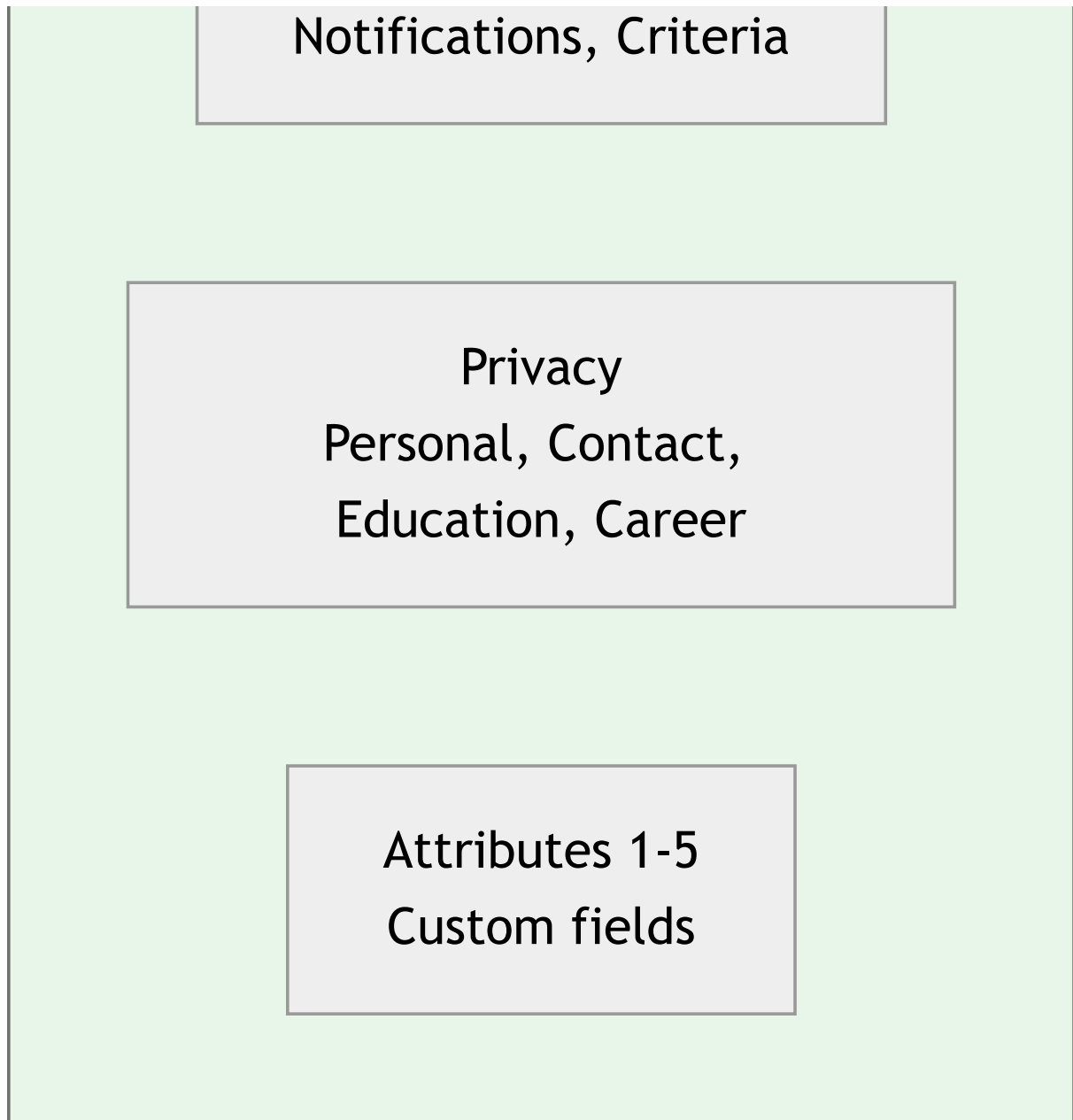
Education

School → University →
Postgraduate → PhD
Training & Courses,
Specialties

Career

History, Profession, Title,
Field, Industry

Vacancy Settings



Master Data Fields:

Category	Fields
Personal	firstName, lastName, fullName, gender, dateOfBirth, nationality1-4, residenceCountry, city, address1-3
Documents	nationalId, passport1-4, drivingLicense
Languages	motherTongue, language1-4 (with proficiency)
Contacts	phoneNumber, email, landlineNumber, linkedIn, facebook, whatsApp
Education	school, university, postgraduate, phd, trainingAndCourses[], specialtiesAndSkills[]
Career	careerHistory[], profession, title, field, industry, careerCountry
Vacancy	vacancyNotificationEnabled, vacancyCriteria{}
Privacy	privacyPersonalInfo, privacyContactInfo, privacyEducation, privacyCareer

2. Legal Entity (الشخصية الاعتبارية)

The **organization** — a corporate identity managed by admins.



Legal Entity

Corporate Info

Formal Name, Commercial
Name, Search Name
Domain Name, Names 1-5

Legal Structure

Org Level:
Holding/Individual/Branch
Org Type, Legal Entity
Type, Date of Operation

Registration

Country, City, Registration
Address
Headquarter Address

Required Fields

Operations
Address, District, Country,
Region
Language, Second
Language, Timezone

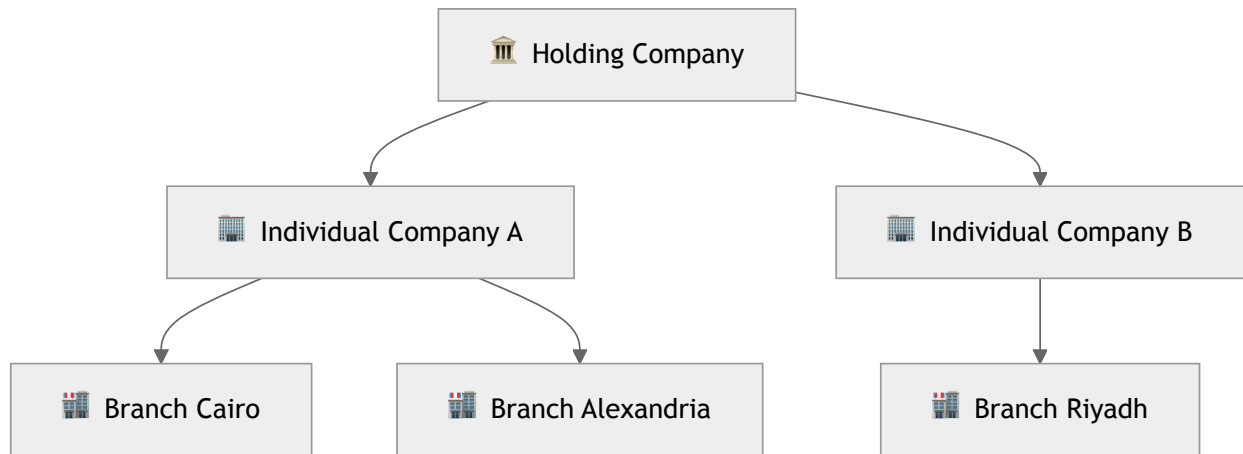
Registration Docs
Commercial Reg., Tax Card
Manufacturing Reg., VAT
Number

Contacts
Email, Website, Phone,
Mobile, Fax





Organization Levels:



3. Virtual Character (الشخصية الافتراضية)

The **position** — a role within an organization that can be linked to a natural person.



Virtual Character (Position)

Position Info

Name, Names 1-3,
Description, Code

Languages 1-4

Contact

Mobile, Telephone, Email

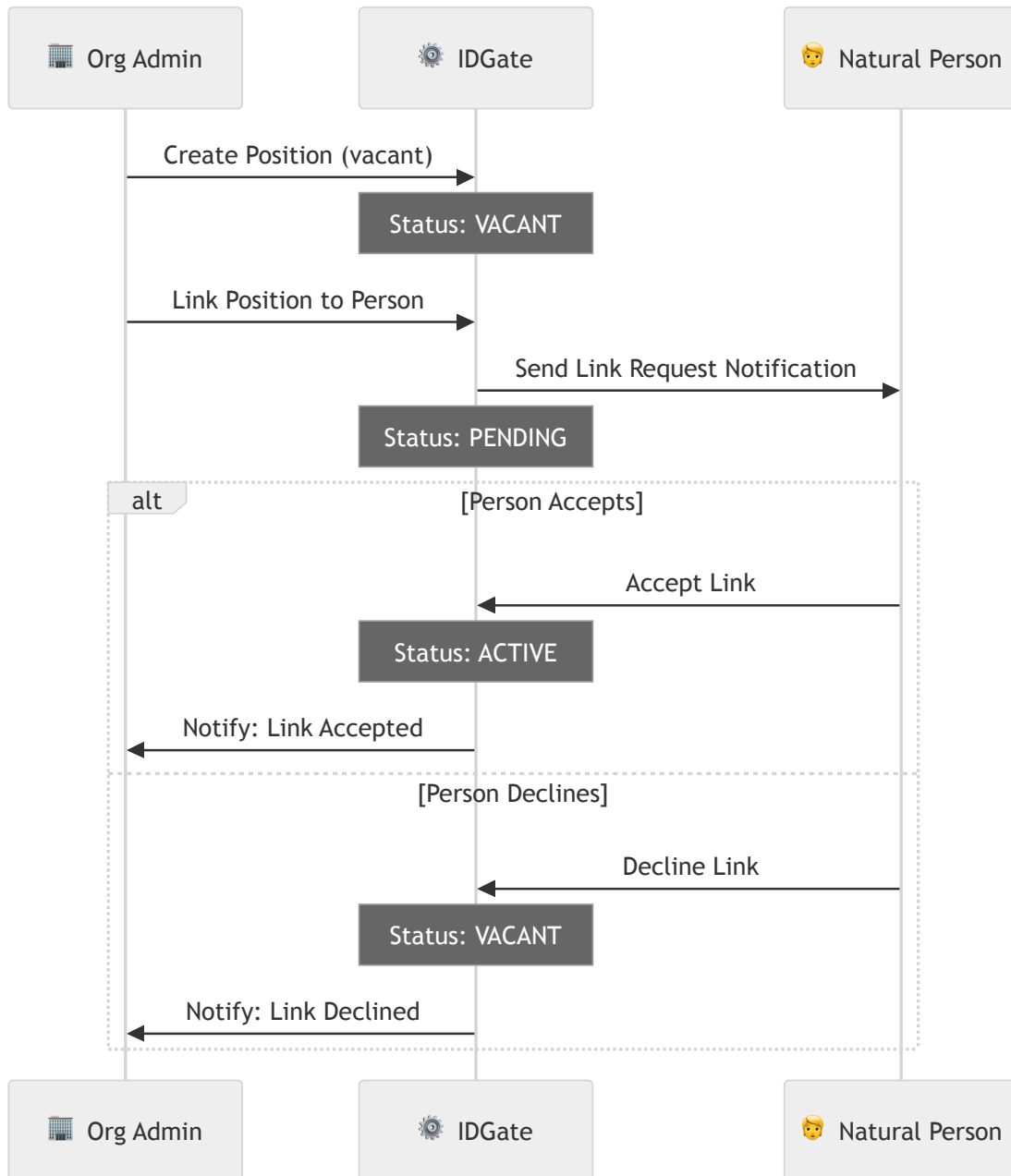
Structure Assignment

Org Node, Management
Node

Function Node,
Geographical Node

Authorization
Position Profile, Delegation
Subjects
Delegation Limits, Can
Delegate Others
Duration, Start/End Dates

Linking
Linked Natural ID, Link
Status
Display History, Location
Privacy

Position Linking Flow:

Organization Structure System

Each organization can define **4 independent structure hierarchies**, each with up to **3 levels**:



Organization

4 Geographical Structure (Locations)

Level 1: Country

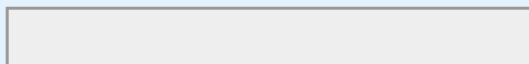


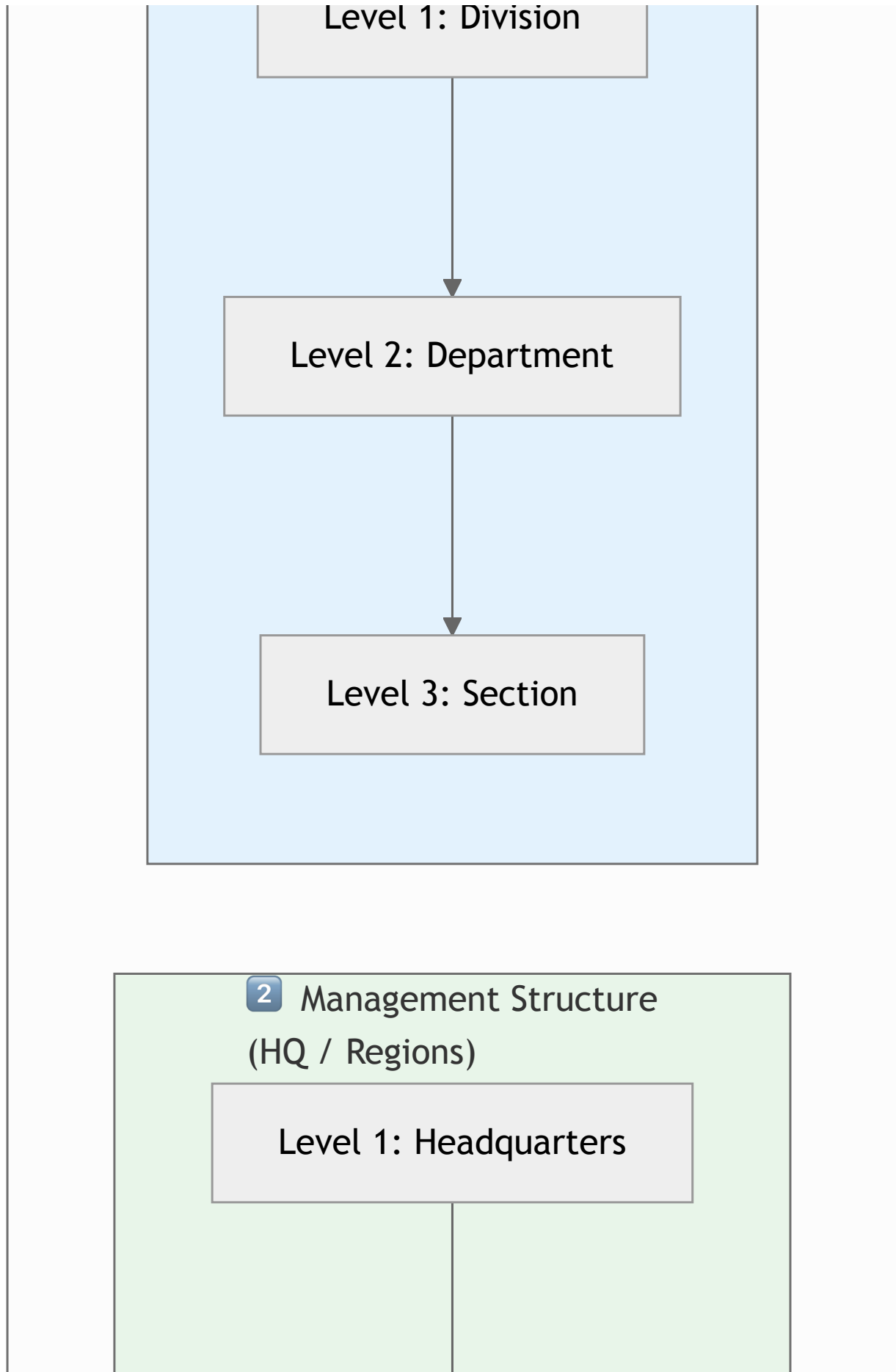
Level 2: Governorate/State

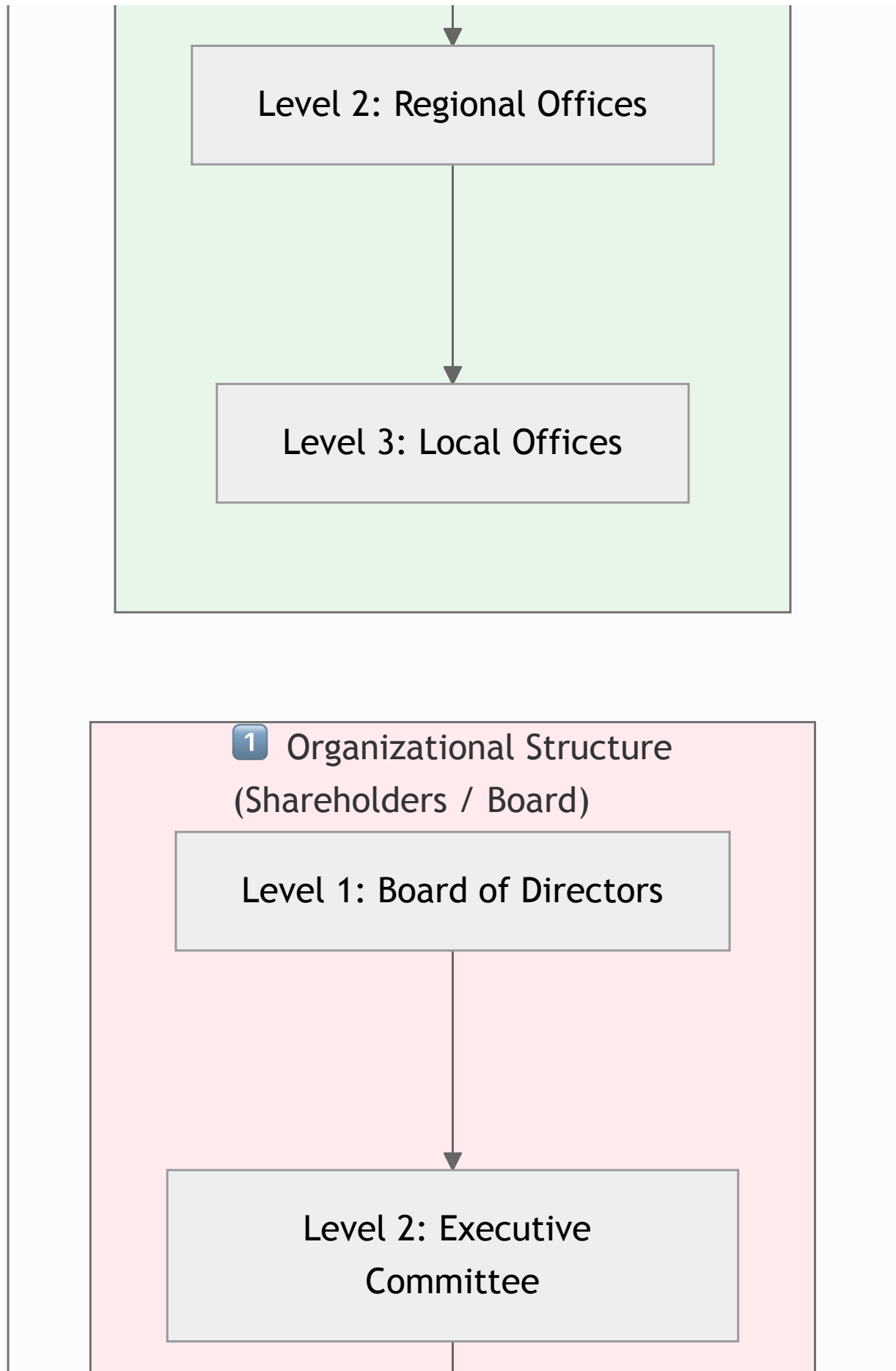


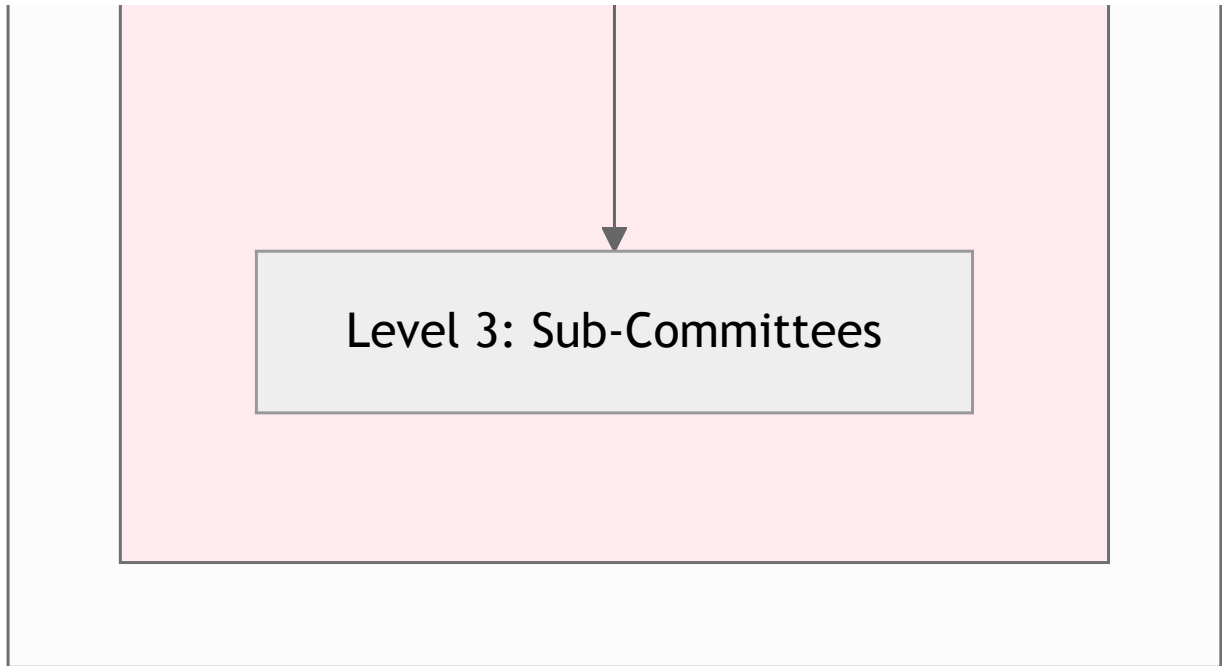
Level 3: District/City

3 Function Structure (Departments)



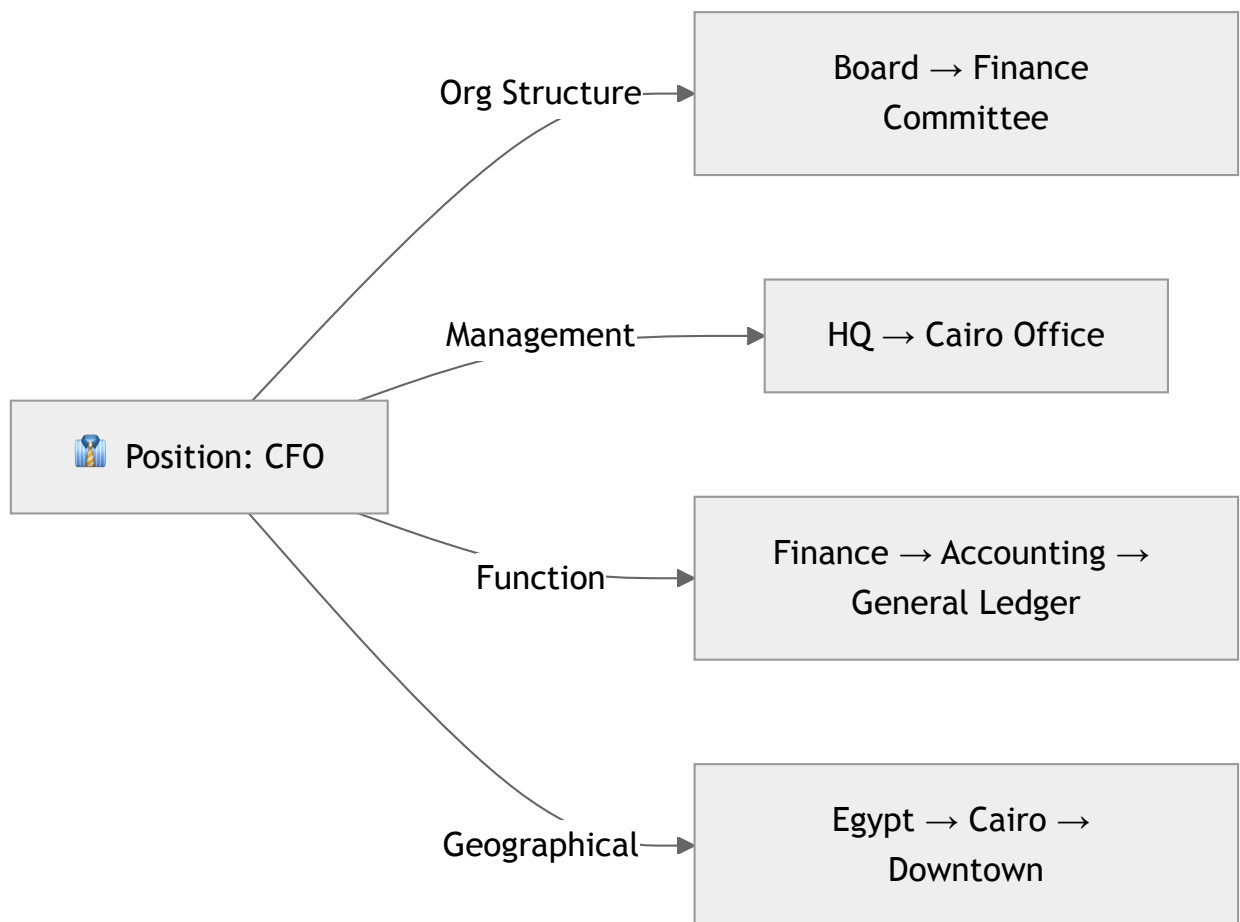




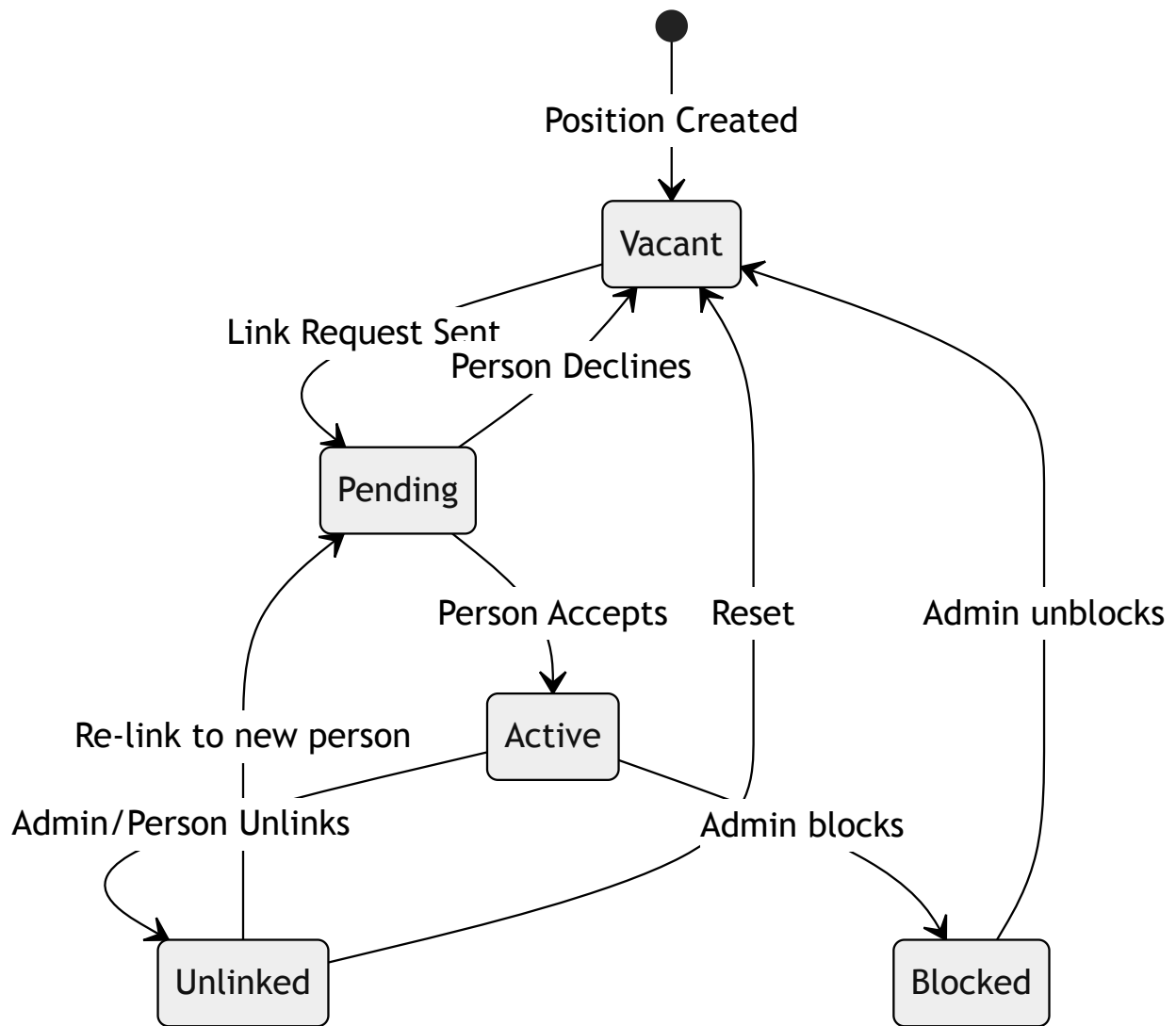


Position Assignment

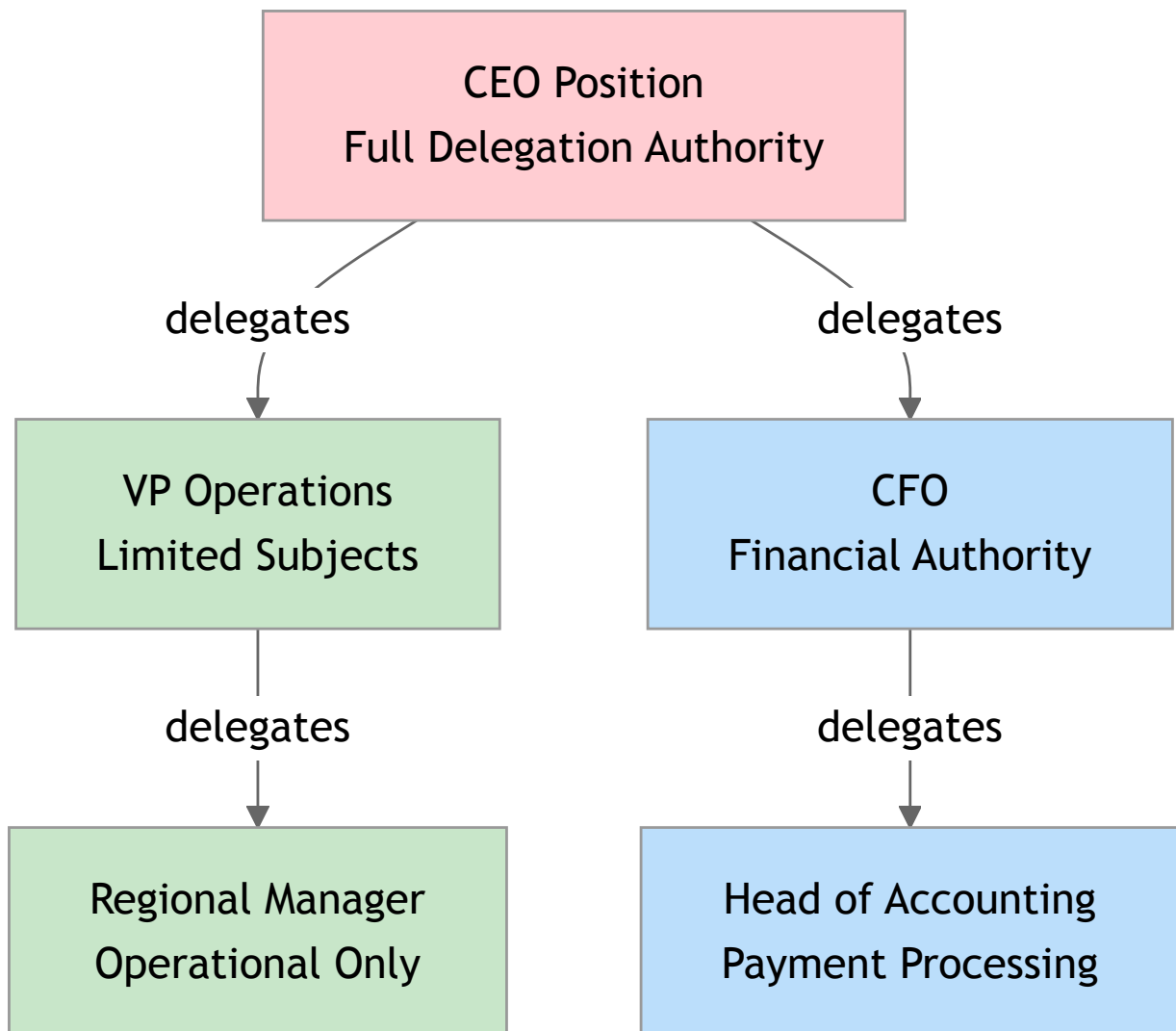
A position can be assigned to **one node in each structure type**:



Position Lifecycle



Authorization Model

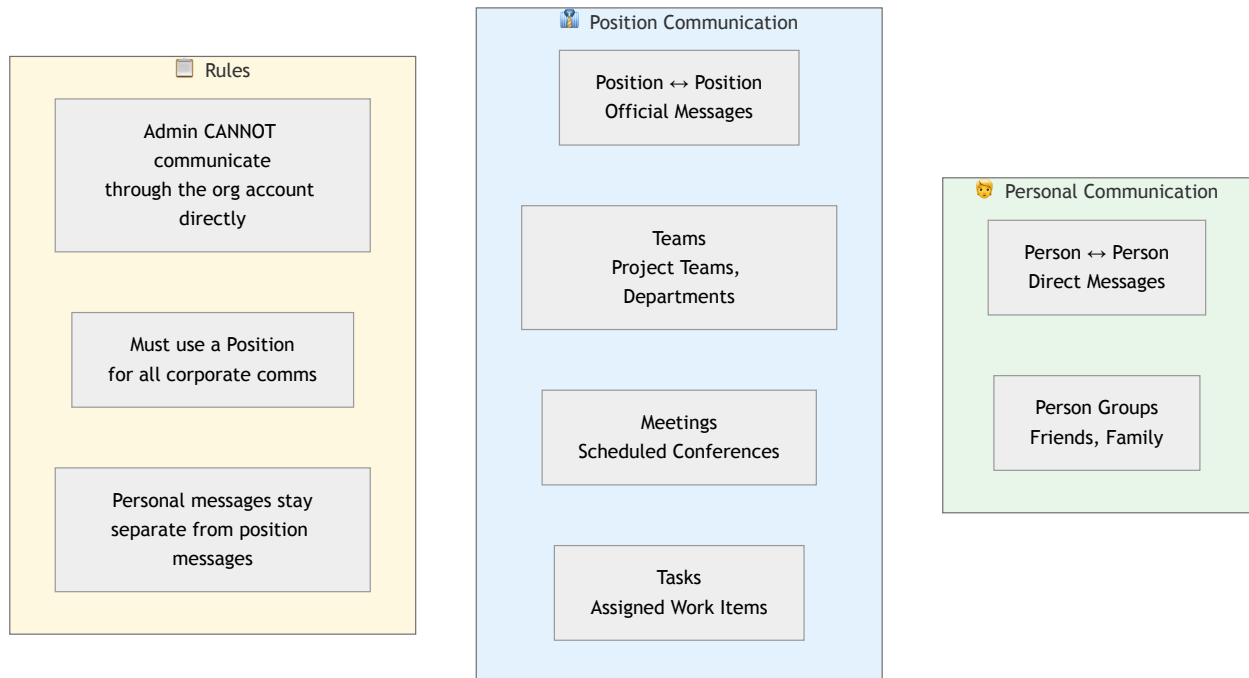


Each position defines:

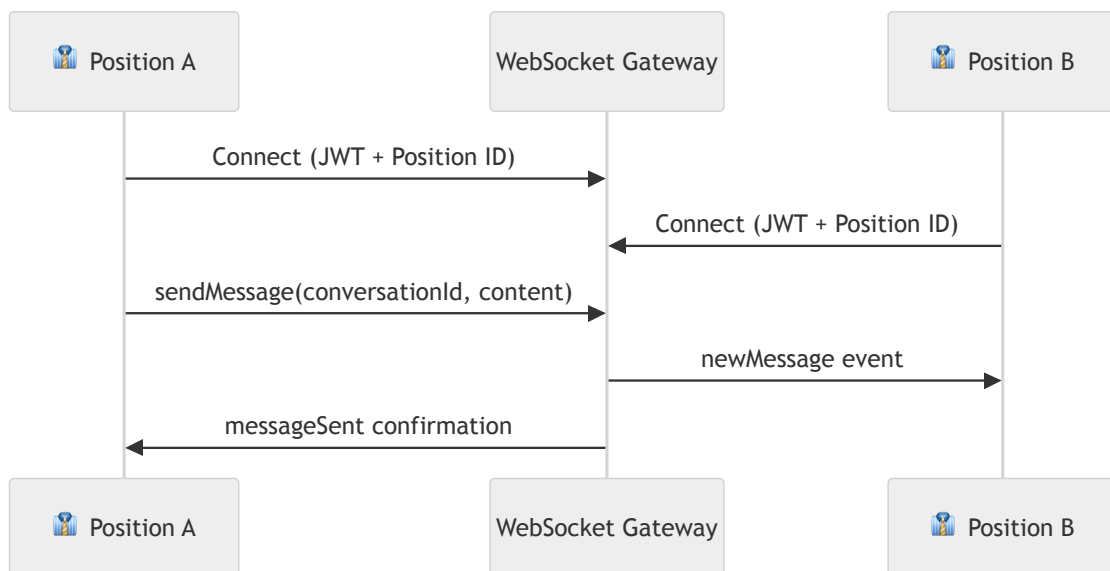
- **positionProfile** — what system features it can access
- **delegationSubjects** — what business authorities it holds
- **delegationLimits** — monetary/quantity limits per subject
- **canDelegateOthers** — whether it can pass authority downstream
- **delegationDuration** — open-ended or time-limited

Communication Model

IDGate separates communication by identity type:



Real-Time Communication



Privacy & Authorization

Privacy Levels

Each data category can be set to one of three privacy levels:

Privacy Levels

 **Public**
Visible to everyone

 **Contacts**
Visible to contacts only

 **Closed**
Visible to owner only

Natural Character Privacy Controls

Category	Controls	Default
Personal Info	Name, gender, DOB, nationality, address	Public
Contact Info	Phone, email, social media	Public
Education	Degrees, courses, skills	Public
Career	History, profession, industry	Public

Legal Entity Privacy Controls

Category	Controls	Default
Corporate Info	Names, legal type, registration	Public
Contact Info	Phone, email, website	Public
Field of Operation	Industry, brands, products	Public
Structure Info	Org structure, positions	Contacts

API Reference

Authentication

Method	Endpoint	Description
POST	<code>/auth/register</code>	Register new account
POST	<code>/auth/login</code>	Login with phone + password
POST	<code>/auth/verify-otp</code>	Verify OTP code
POST	<code>/auth/refresh</code>	Refresh access token
POST	<code>/auth/logout</code>	Logout (invalidate tokens)

Profile

Method	Endpoint	Description
GET	<code>/entities/profile/me</code>	Get my full profile
PUT	<code>/entities/profile/me</code>	Update my profile
GET	<code>/entities/profile/:id</code>	Get user public profile
GET	<code>/entities/search/users?q=</code>	Search users

Organizations

Method	Endpoint	Description
GET	<code>/entities/organizations/me</code>	My organizations
POST	<code>/entities/organizations</code>	Create organization
GET	<code>/entities/organizations/search?q=</code>	Search organizations
GET	<code>/entities/organizations/:id</code>	Get organization details
PUT	<code>/entities/organizations/:id</code>	Update organization

Positions

Method	Endpoint	Description
GET	/entities/positions/me	My active positions
GET	/entities/positions/pending	Pending link requests
POST	/entities/organizations/:orgId/positions	Create position
GET	/entities/organizations/:orgId/positions	List org positions
GET	/entities/positions/:id	Get position details
PUT	/entities/positions/:id	Update position
POST	/entities/positions/:id/link	Link to person
POST	/entities/positions/:id/accept-link	Accept link request
POST	/entities/positions/:id/decline-link	Decline link request
POST	/entities/positions/:id/unlink	Unlink from person

Organization Structure

Method	Endpoint	Description
GET	/entities/organizations/:orgId/structure?type=	Get structure tree
POST	/entities/organizations/:orgId/structure	Create node
PUT	/entities/structure/:nodeId	Update node
DELETE	/entities/structure/:nodeId	Delete node

Notifications

Method	Endpoint	Description
GET	/notifications?page=&limit=	Get notifications
GET	/notifications/unread-count	Unread count
POST	/notifications/:id/read	Mark as read
POST	/notifications/read-all	Mark all as read
POST	/notifications/:id/action?result=	Action a notification

Communication

Method	Endpoint	Description
GET	/communication/conversations	List conversations
POST	/communication/conversations	Create conversation
GET	/communication/conversations/:id	Get conversation
GET	/communication/conversations/:id/messages	Get messages
POST	/communication/conversations/:id/messages	Send message

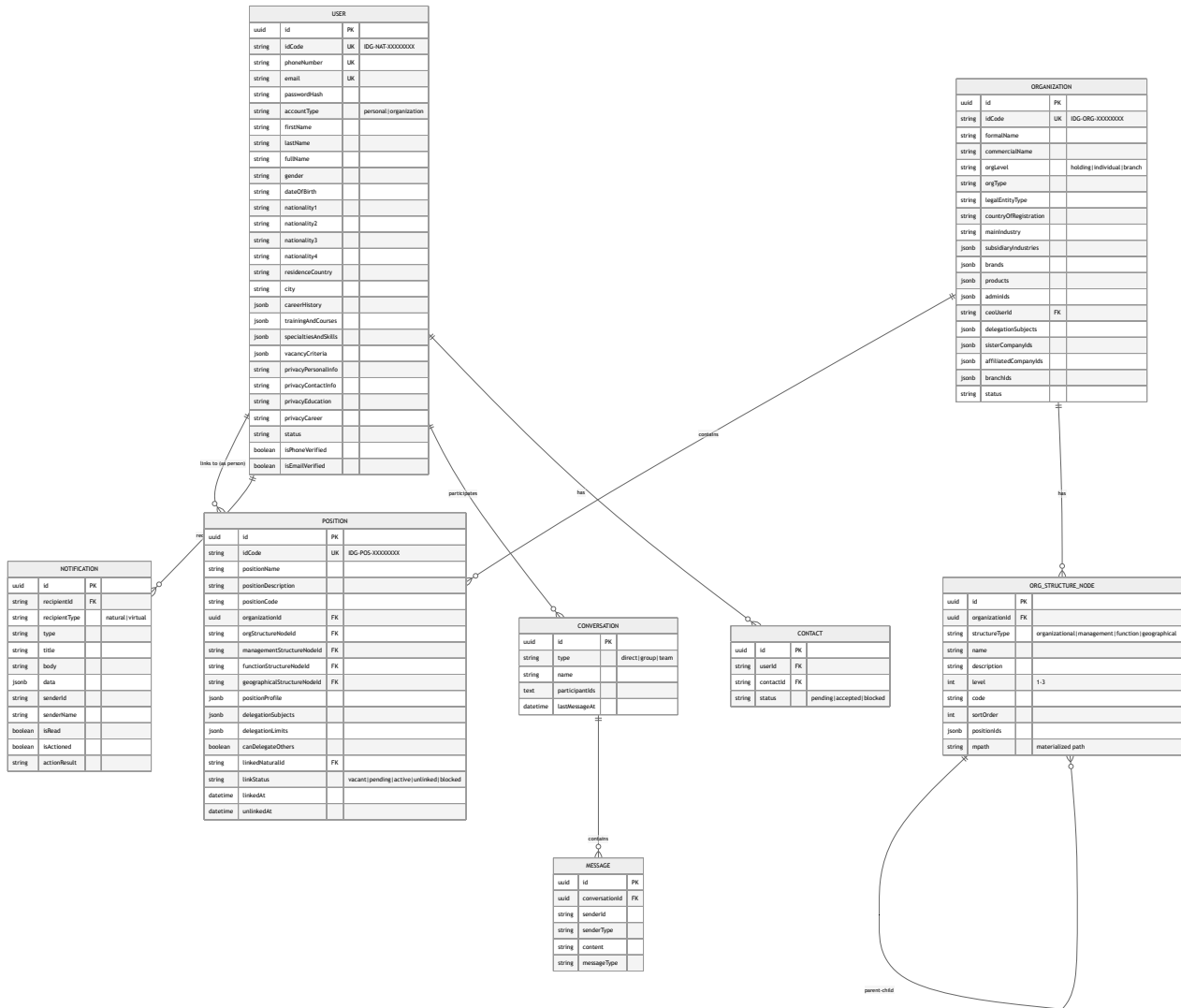
Contacts

Method	Endpoint	Description
GET	/contacts	Get my contacts
POST	/contacts/request	Send contact request
POST	/contacts/:id/accept	Accept request
POST	/contacts/:id/reject	Reject request
DELETE	/contacts/:id	Remove contact



Database Schema

Entity Relationship Diagram



Project Structure

```

id-gate/
├── backend/                                # NestJS REST + WebSocket API
│   ├── src/
│   │   ├── main.ts                        # App bootstrap
│   │   ├── app.module.ts                 # Root module
│   │   ├── auth/                          # Authentication
│   │   │   ├── auth.module.ts
│   │   │   ├── auth.service.ts          # Login, Register, JWT
│   │   │   ├── auth.controller.ts
│   │   │   ├── jwt-auth.guard.ts
│   │   │   ├── jwt.strategy.ts
│   │   │   ├── dto/
│   │   │   │   ├── register.dto.ts
│   │   │   │   └── login.dto.ts
│   │   │   └── entities/
│   │   │       ├── user.entity.ts       # Natural Character entity
│   │   │       └── entities/            # Core business entities
│   │   │           ├── entities.module.ts
│   │   │           ├── entities.service.ts # CRUD for all entity types
│   │   │           ├── entities.controller.ts
│   │   │           └── entities/
│   │   │               ├── organization.entity.ts # Legal Entity
│   │   │               ├── position.entity.ts    # Virtual Character
│   │   │               └── org-structure-node.entity.ts # Structure tree
│   │   ├── communication/                # Messaging system
│   │   │   ├── communication.module.ts
│   │   │   ├── communication.service.ts
│   │   │   ├── communication.controller.ts
│   │   │   ├── communication.gateway.ts # Socket.IO WebSocket
│   │   │   └── entities/
│   │   │       ├── conversation.entity.ts
│   │   │       └── message.entity.ts
│   │   ├── contacts/                     # Contact management
│   │   │   ├── contacts.module.ts
│   │   │   ├── contacts.service.ts
│   │   │   ├── contacts.controller.ts
│   │   │   └── entities/
│   │   │       └── contact.entity.ts
│   │   ├── notifications/                # Push & in-app notifications
│   │   │   ├── notifications.module.ts
│   │   │   ├── notifications.service.ts
│   │   │   ├── notifications.controller.ts
│   │   │   └── entities/
│   │   │       └── notification.entity.ts

```

```

├── feed/                                # Activity feed
│   ├── feed.module.ts
│   ├── feed.service.ts
│   └── feed.controller.ts
├── teams/                               # Teams & groups
│   └── teams.module.ts
├── seed/                                # Demo data seeder
│   ├── seed.module.ts
│   ├── seed.service.ts
│   └── seed.controller.ts
├── package.json
├── tsconfig.json
├── nest-cli.json
├── mobile/                             # Expo React Native App
│   ├── app/                            # Expo Router (file-based routing)
│   │   ├── _layout.tsx                # Root layout
│   │   ├── index.tsx                  # Entry redirect
│   │   ├── (auth)/                    # Auth screens (unauthenticated)
│   │   │   ├── register.tsx
│   │   │   ├── login.tsx
│   │   │   └── verify-otp.tsx
│   │   ├── (tabs)/                    # Main tab navigation
│   │   │   ├── home.tsx               # Dashboard
│   │   │   ├── messages.tsx           # Conversations
│   │   │   ├── notifications.tsx      # Notification center
│   │   │   ├── tools.tsx              # Business tools
│   │   │   └── settings.tsx           # Profile & settings
│   │   ├── chat/                      # Chat screens
│   │   └── organizations/             # Organization management
│   │       ├── index.tsx               # List organizations
│   │       ├── [id].tsx                # Organization details
│   │       └── add.tsx                 # Join/create organization
│   ├── src/
│   │   ├── services/                  # API client layer
│   │   │   ├── api.ts                 # Axios instance
│   │   │   ├── auth.service.ts
│   │   │   ├── entity.service.ts
│   │   │   ├── communication.service.ts
│   │   │   └── feed.service.ts
│   │   ├── stores/                    # Zustand state management
│   │   │   └── auth.store.ts
│   │   ├── config/                    # App configuration
│   │   └── utils/                      # Utilities
│   ├── package.json
│   └── app.json
├── shared/                             # Shared TypeScript types
└── src/

```

```

├── index.ts
├── types/
│   ├── entities.ts           # Entity interfaces
│   ├── api.ts               # API request/response types
│   ├── communication.ts     # Chat types
│   └── business-tools.ts    # Tool types
├── package.json
└── tsconfig.json

├── docker-compose.yml       # Local dev infrastructure
├── package.json             # Workspace root
└── README.md

```

Tech Stack

Backend

Technology	Purpose	Version
NestJS	Application framework	~10.4
TypeORM	ORM / Database access	~0.3.20
PostgreSQL	Primary database	16
Redis	Caching & pub/sub	7
Socket.IO	Real-time WebSocket	via @nestjs/platform-socket.io
Passport + JWT	Authentication	passport-jwt 4
Swagger	API documentation	@nestjs/swagger 8
class-validator	DTO validation	~0.14
bcrypt	Password hashing	~5.1
Helmet	Security headers	~8.0

Mobile

Technology	Purpose	Version
Expo	React Native framework	SDK 54
Expo Router	File-based navigation	~6
React Native	UI framework	0.76+
Zustand	State management	~5
Axios	HTTP client	~1.7
React Query	Server state	@tanstack/react-query 5
Socket.IO Client	Real-time messaging	—

Infrastructure

Technology	Purpose
Docker Compose	Local development
Railway	Backend hosting (production)
Vercel	Mobile web hosting (production)
MinIO	Local S3-compatible file storage
GitHub	Source control

Getting Started

Prerequisites

- **Node.js** 20+
- **Docker** & Docker Compose
- **Expo CLI** (`npx expo`)

1. Clone & Install

```
git clone https://github.com/mhisham744/id-gate.git
cd id-gate
npm install
```

2. Start Infrastructure

```
docker-compose up -d
```

This starts:

- PostgreSQL on port 5432
- Redis on port 6379
- MinIO on ports 9000 (API) and 9001 (console)

3. Start Backend

```
cd backend
cp .env.example .env # Configure your environment
npm run start:dev
```

Backend starts on `http://localhost:3000` Swagger docs at `http://localhost:3000/api`

4. Seed Demo Data

```
curl -X POST http://localhost:3000/seed
```

Creates:

- Demo user: +201001234567 / Password123!
- Demo organization: IDGate Demo Corp
- Demo position: CEO (linked to demo user)

- Sample org structure

5. Start Mobile

```
cd mobile
npx expo start
```

Options:

- Press `w` for web browser
- Press `i` for iOS simulator
- Press `a` for Android emulator
- Scan QR with Expo Go app

Deployment

Production URLs

Service	URL
Backend API	https://backend-production-2b386.up.railway.app
Mobile Web	https://dist-tau-one-28.vercel.app
API Docs	https://backend-production-2b386.up.railway.app/api

Deploy Backend (Railway)

```
cd backend
# Railway auto-deploys from GitHub main branch
git push origin main
```

Deploy Mobile (Vercel)

```
cd mobile
npx expo export --platform web
# Deploy dist/ to Vercel
```



Environment Variables

Backend (.env)

```
# Database
DATABASE_URL=postgresql://user:pass@host:5432/idgate
# OR individual settings:
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=idgate
DB_PASSWORD=idgate_dev_password
DB_DATABASE=idgate

# JWT
JWT_SECRET=your-super-secret-jwt-key
JWT_EXPIRES_IN=7d

# App
NODE_ENV=development
PORT=3000

# Redis (optional)
REDIS_URL=redis://localhost:6379

# File Storage (optional)
S3_ENDPOINT=http://localhost:9000
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_BUCKET=idgate
```

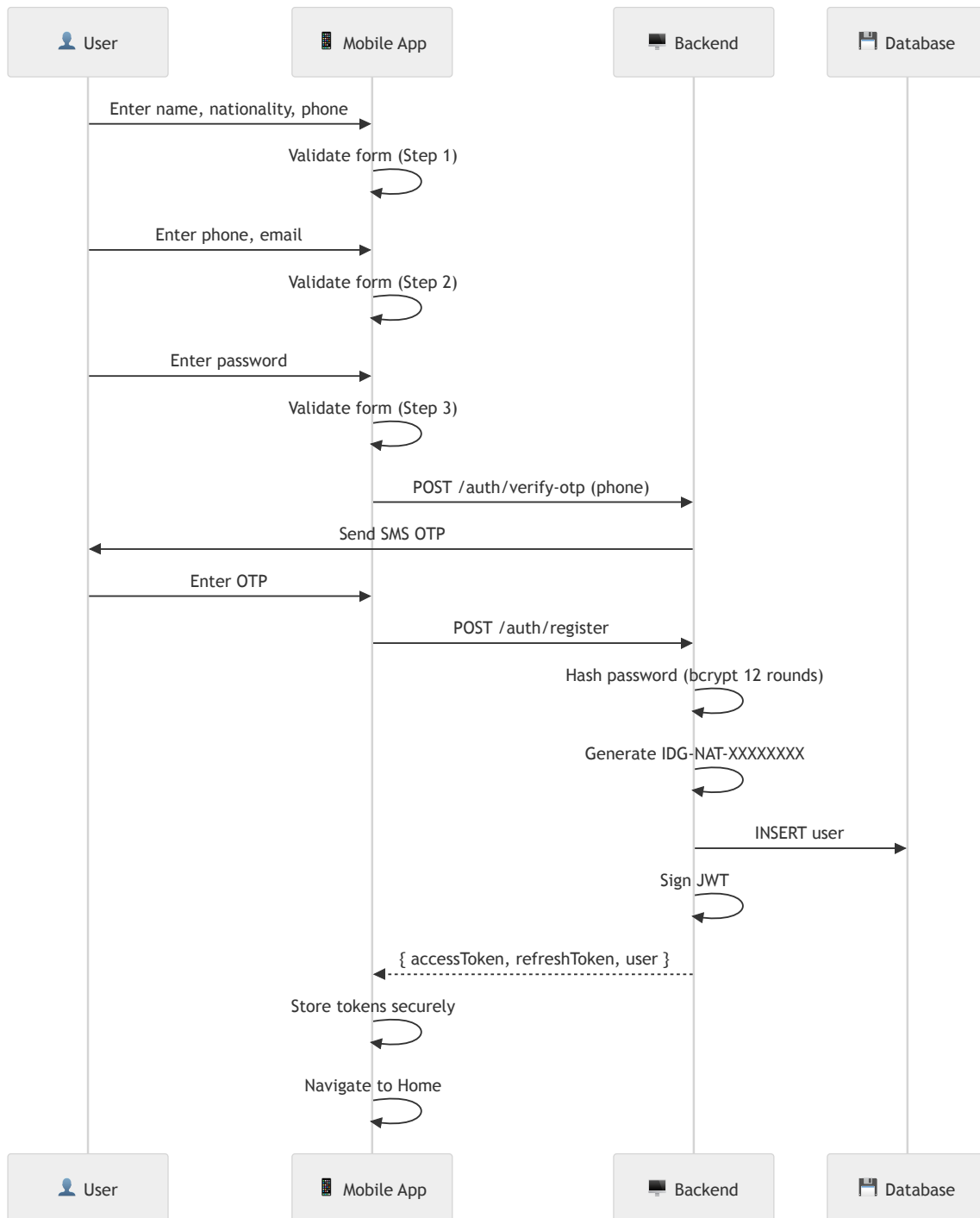
Mobile (src/config)

The mobile app reads the backend URL from its config:

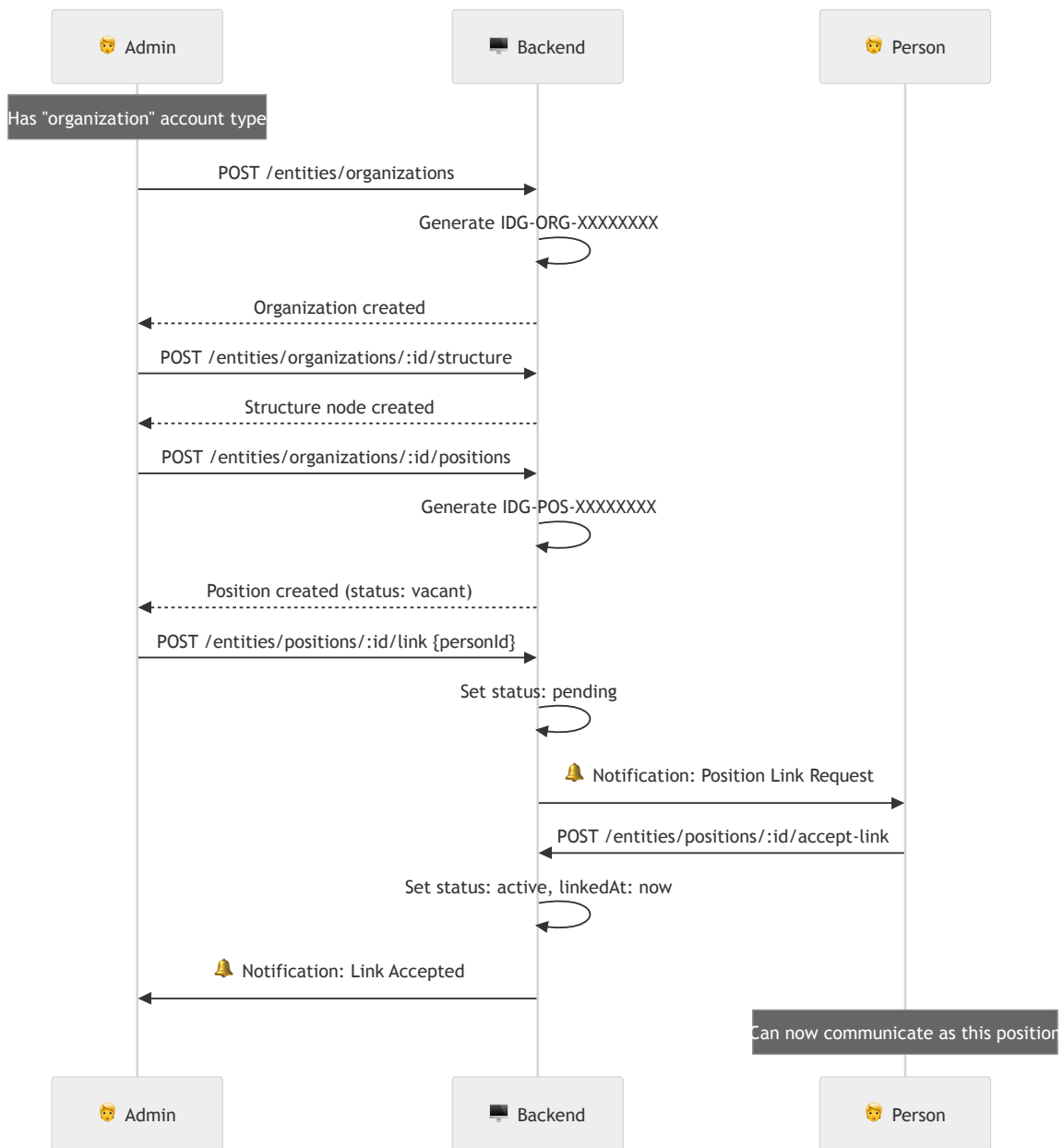
```
export const config = {  
  api: {  
    baseUrl: 'https://backend-production-2b386.up.railway.app',  
  },  
};
```

System Flow Diagrams

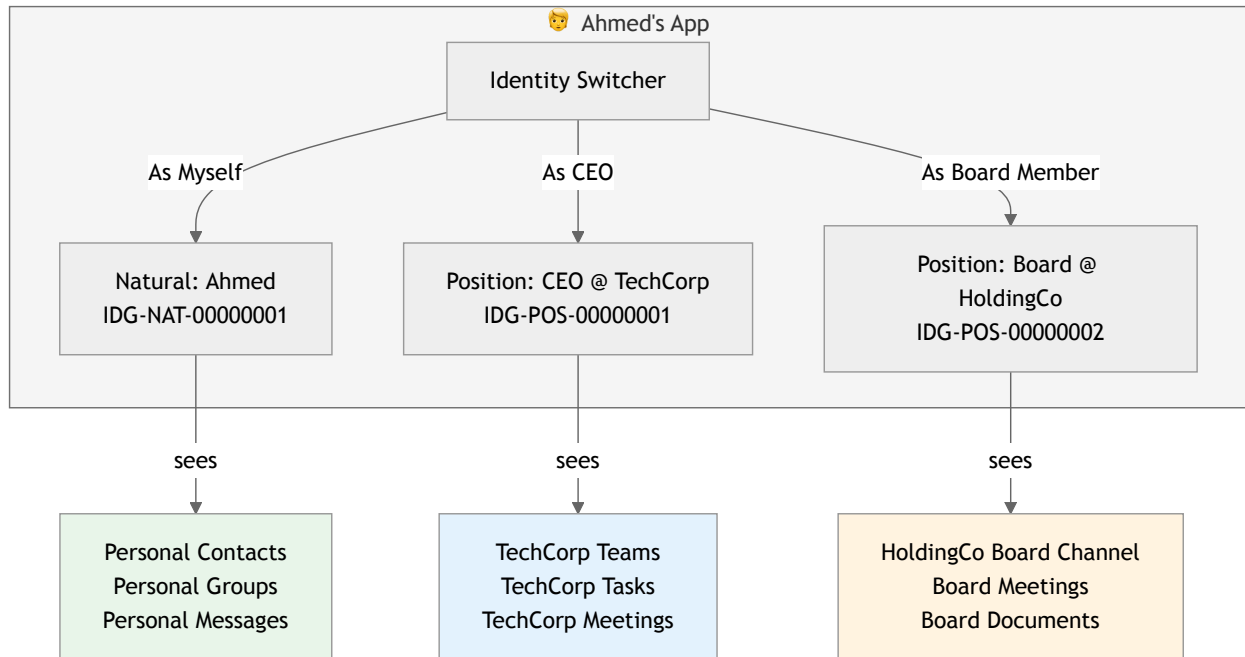
Complete Registration Flow



Organization Creation & Position Linking

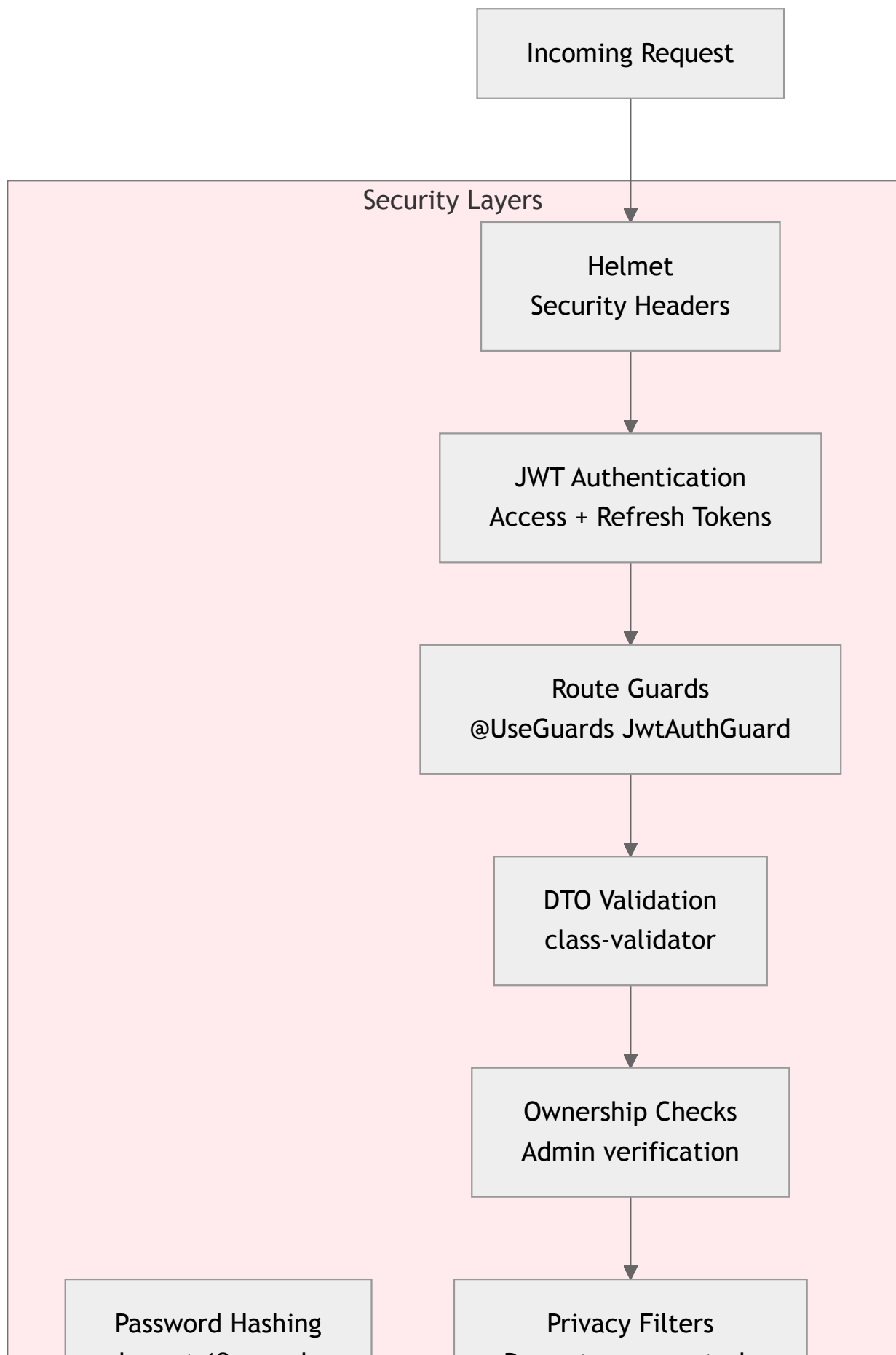


Identity Switching During Communication





Security Architecture





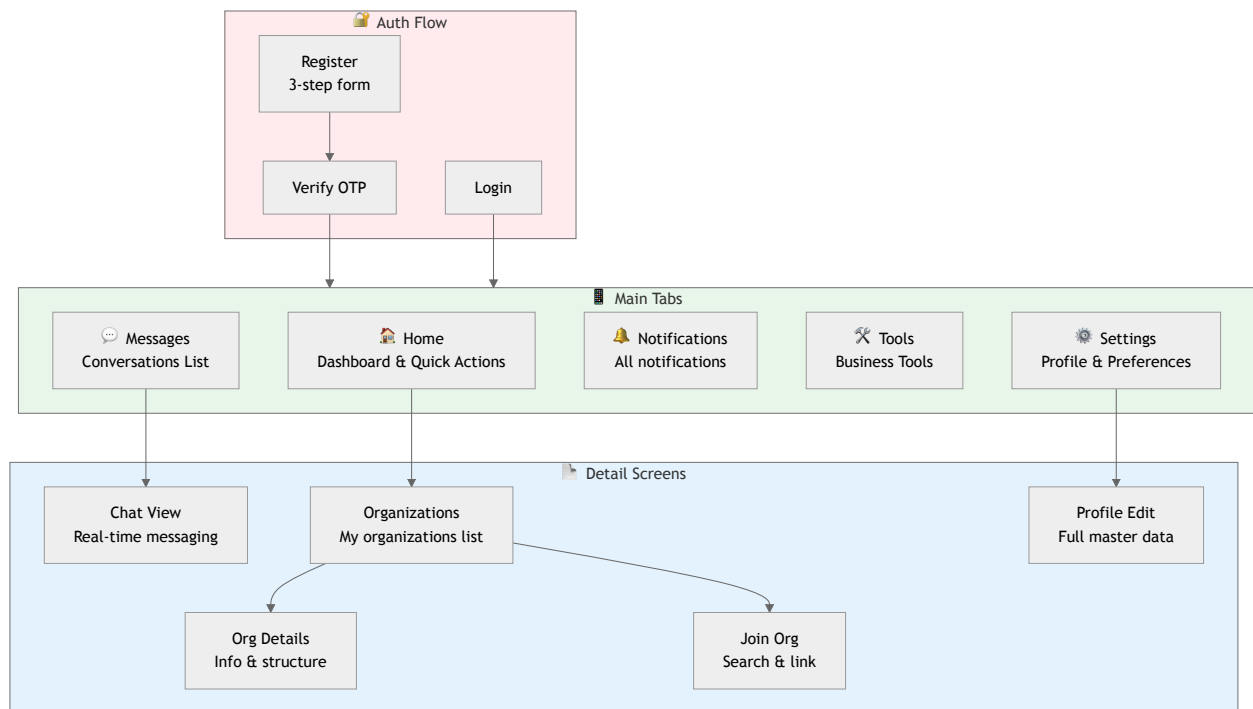
Key Security Features:

- JWT tokens with short expiry + refresh token rotation
- bcrypt password hashing (12 salt rounds)
- Helmet security headers
- Input validation on all endpoints
- Ownership verification (only admins can modify their orgs)
- Privacy-filtered public profiles
- SQL injection prevention via TypeORM parameterized queries

Notification Types

Type	Trigger	Action Required
contact_request	Someone wants to add you	Accept / Decline
position_link_request	Organization wants to link you	Accept / Decline
meeting_request	Meeting invitation	Accept / Decline
task_assignment	Task assigned to your position	Acknowledge
delegation_request	Authority delegation offered	Accept / Decline
message	New message in conversation	Read
system	System announcement	Read

Mobile App Screens



Testing

```

# Backend unit tests
cd backend && npm test

# Backend e2e tests
cd backend && npm run test:e2e

# Mobile lint
cd mobile && npm run lint

# Type checking (both)
cd backend && npx tsc --noEmit
cd mobile && npx tsc --noEmit
  
```

License

Private — All rights reserved.

IDGate — Your Identity, Your Control 

(Identification Gate)

An identity-verified communication platform where people communicate through their official positions and credentials in a verified, secure way.

Core Concept

IDGate introduces a three-entity identity model:

1. **Natural Character** (الشخصية الطبيعية) - The real person with a permanent account
2. **Virtual Character / Position** (الشخصية الافتراضية) - Role-based identity granted by organizations, only active when linked to a natural person
3. **Legal Entity** (الشخصية الاعتبارية) - Organizations that create positions and link them to people

Key Differentiators

- **Official communication** - Unlike WhatsApp/Facebook, communication carries verified identity weight
- **Position-based identity** - People communicate through their verified organizational roles
- **Organization structure** - Full org hierarchy support (departments, positions, levels)
- **Integrated business services** - Meetings, tasks, calendar, projects in one platform
- **Security through verification** - Positions only activate when linked to verified natural persons

Architecture

```
id-gate/
├─ mobile/      # React Native mobile app (iOS & Android)
├─ backend/     # NestJS backend API
├─ shared/      # Shared TypeScript types & schemas
└─ docs/        # Documentation
```

Tech Stack

- **Mobile:** React Native + TypeScript + Expo
- **Backend:** NestJS + TypeScript + PostgreSQL + Redis
- **Real-time:** Socket.IO for messaging
- **Auth:** JWT + OTP verification
- **Storage:** AWS S3 / MinIO for media

Modules

MVP Phase 1: Identity & Communication

- User registration & identity verification
- Natural character profiles
- Legal entity (organization) creation
- Position creation & linking
- Direct messaging (person-to-person, position-to-position)
- Teams, Groups, Broadcast lists
- Address book with approval-based contacts

Phase 2: Business Tools

- Text meetings
- Conferences
- Calendar & scheduling
- Task management

- Project management
- Reminders & notes

Phase 3: Information & Services

- Financial/economic news feeds
- ERP data integration
- Vacancy/recruitment
- Publishing & advertising
- Reports & analytics

Getting Started

```
# Install dependencies
npm install

# Start backend
cd backend && npm run start:dev

# Start mobile app
cd mobile && npx expo start
```

License

Proprietary - All rights reserved.